ASSIGNMENT 1

Student Details	Information
Name	D.SANJANA
Register Number	192471029
Programme / Department	CSE&BIO
Course / Subject	DSA0111
GitHub Repository	https://github.com/sanjjaaana/DSA0111-192471029-Assignment1

Scenario and Design Assumptions

The metro network has three zones. The base fare is Rs 10 for Inner, Rs 20 for Middle and Rs 30 for Outer. A 20% surcharge applies during peak hours: 08:00–09:59 and 17:00–19:59. After a fare is deducted, at least Rs 50 must remain on the card. Students and senior citizens receive a concession, but the assignment does not state the concession percentage or exact eligibility rules.

- Money is stored as integer paise (Rs 1 = 100 paise) so that calculations are exact.
- For the worked examples and test tables, no student/senior concession is applied because its percentage is not specified. A configurable concession percentage can be added when the faculty supplies the value.
- Time boundaries are start-inclusive and end-exclusive: 08:00 and 17:00 are peak; 10:00 and 20:00 are not peak.
- For percentage calculations, integer arithmetic is used. A 20% surcharge is calculated as baseFare * 20 / 100. The selected fares are whole rupees, so the surcharge is exact in paise.
- The minimum remaining balance is checked after calculating the final fare, not before.

TASK 1 – Understand the Problem and Identify the Objects

Object-oriented analysis divides the metro system into objects. Each object stores related data and provides behaviour that operates on that data.

Object	Important data (attributes)	Behaviour (methods)	Reason for object
SmartCard	cardId, balancePaise, activeStatus	checkBalance(), canTravel(), deduct()	Keeps balance rules in one place and prevents an invalid deduction.
Passenger	passengerId, name, category	getCategory(), getConcessionRate()	Represents the traveller and concession category.
Journey	journeyId, zone, hour, date	isPeakHour(), calculateFare()	Represents one trip and its fare conditions.
Zone	zoneId, zoneName, baseFarePaise	getBaseFare()	Represents the fare zone.
FareTable	base fares, surcharge percent, minimum balance	getFare(), calculateFinalFare()	Stores fare rules centrally so a revision is made once.

UML Class Diagram (text representation)

```
Passenger
- passengerId : int
- name : string
- category : PassengerCategory
+ getCategory() : PassengerCategory

SmartCard
- cardId : int
- balancePaise : long long
- active : bool
+ canTravel(farePaise) : bool
+ deduct(farePaise) : bool
+ getBalance() : long long

Journey
- journeyId : int
- zone : Zone
- hour : int
+ isPeakHour() : bool
+ calculateFare() : long long

FareTable
- baseFarePaise[3] : long long
- surchargePercent : int
- minimumBalancePaise : long long
```

```
+ getFare(zone, peak) : long long

Relationships:
Passenger 1 ---- 1 SmartCard
Passenger 1 ---- many Journey
Journey many ---- 1 Zone
Journey ---- uses ---- FareTable
SmartCard ---- validates/deducts ---- Journey fare
```

Procedural and OOP approaches

In a procedural design, the program uses separate variables and free functions such as balance, baseFare(), isPeakHour() and deductFare(). All functions must carefully use the same balance variable. In an OOP design, SmartCard owns its balance and FareTable owns fare rules. The data and the operations related to them are grouped together.

Four OOP principles

- Encapsulation: SmartCard keeps balancePaise private. Other code must call deduct() rather than directly changing the balance.
- Abstraction: the gate calls calculateFare() and deduct(); the user does not need to know every arithmetic step.
- Inheritance: StudentPassenger and SeniorPassenger may inherit common fields and methods from Passenger.
- Polymorphism: each passenger category can override getConcessionRate() or calculateConcession() to apply its own approved concession.

Rules for a valid journey

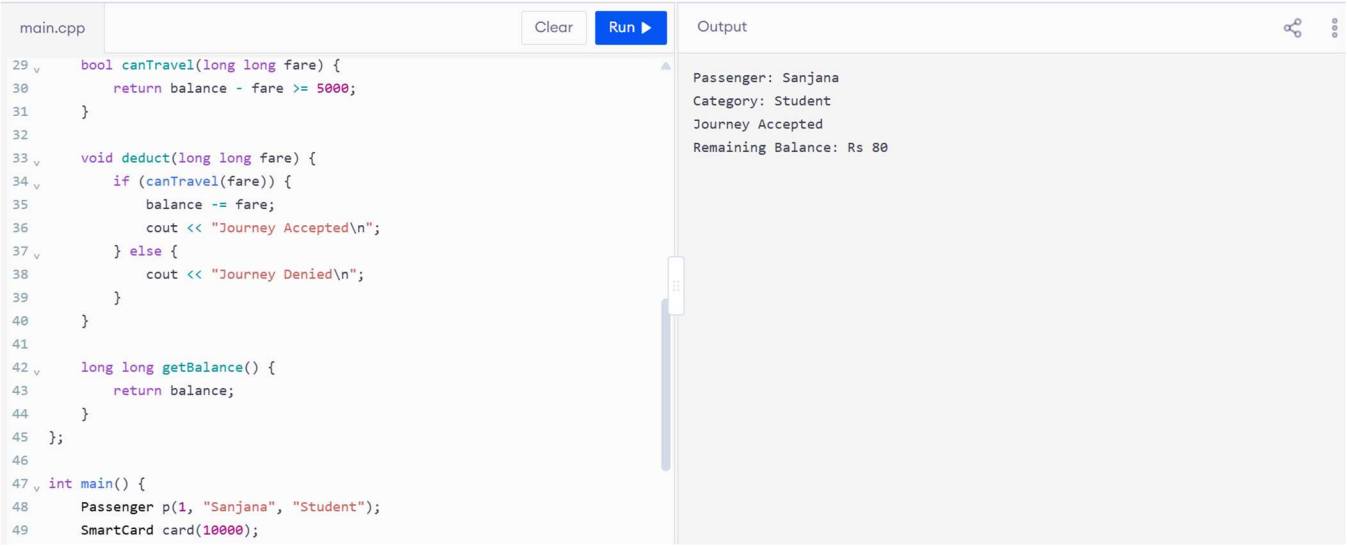
- Zone must be one of the defined zones.
- Hour must be from 0 to 23.
- Peak surcharge applies only for 08:00–09:59 and 17:00–19:59.
- Fare must not be negative.
- The card must be active.
- After deduction, balance must be at least Rs 50 (5000 paise).
- A failed journey must not change the balance.
- Money calculations must remain exact; do not use floating-point rupees for stored balances.

Example: a card with Rs 100 and a Rs 30 fare can travel; its remaining balance is Rs 70. A card with Rs 60 and a Rs 20 fare cannot travel because only Rs 40 would remain.

Alternative concept and complexity

A procedural version can use global variables and free functions. Both procedural and OOP versions take O(1) time for one fare calculation and O(1) additional memory for one card. The main difference is maintenance: when a rule changes, OOP usually localises the change in FareTable or SmartCard, while procedural code may require changes in several functions.

Assumption: the assignment gives no concession percentage. It must be confirmed rather than invented.



TASK 2 – Tokens, Keywords, Identifiers and Constants

C++ tokens are the smallest meaningful elements in a program.

Token kind	Example	Meaning
Keyword	class, int, return, if	Reserved words with fixed C++ meaning.
Identifier	SmartCard, balancePaise, fare	Names created by the programmer.
Constant	20, 5000, 1000	Fixed values used by the program.
Operator	+ , -, *, /, >=, &&	Performs calculation or comparison.
Punctuator	; , { } ( ) ,	Separates statements and groups code.

Recommended constants

```
constexpr long long INNER_FARE_PAISE = 1000;
constexpr long long MIDDLE_FARE_PAISE = 2000;
constexpr long long OUTER_FARE_PAISE = 3000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

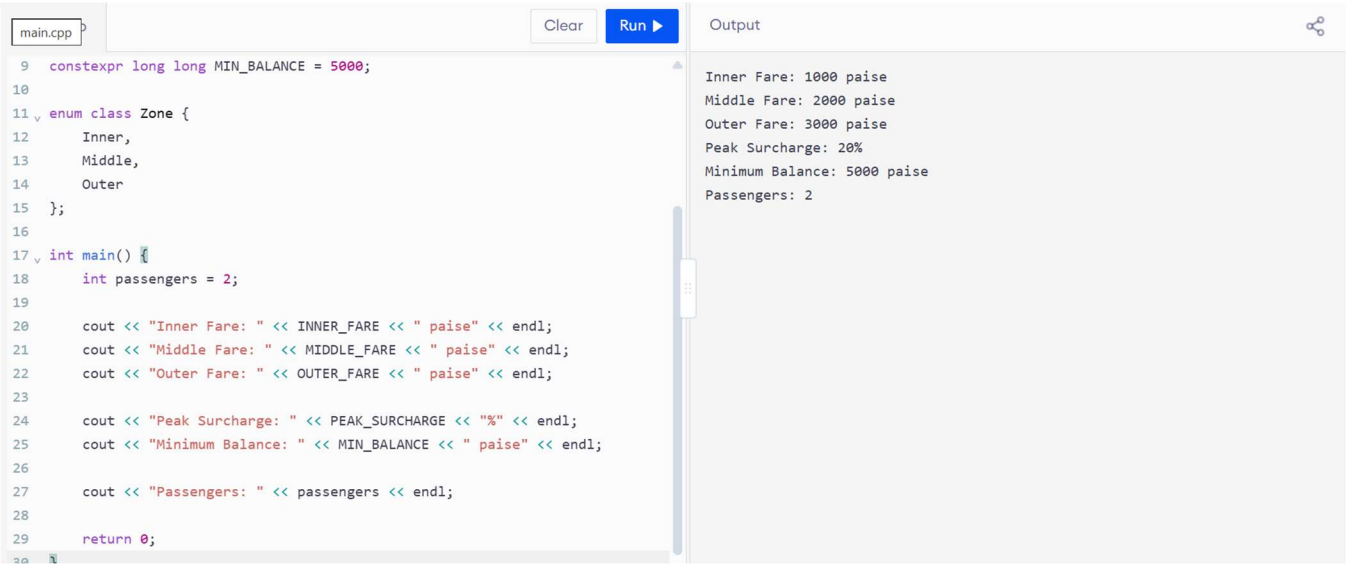
enum class Zone { Inner, Middle, Outer };
```

A #define macro performs text substitution and has no normal C++ type or scope. A const variable is read-only after initialisation. A constexpr value is intended to be evaluated at compile time when possible and is type-checked. constexpr is preferred for fixed fare values and limits. enum class is safer than an unscoped enum because its names are scoped and do not implicitly convert to int.

Identifier conventions: use meaningful names, begin with a letter or underscore, do not use spaces, do not use reserved keywords, and use consistent camelCase or UPPER_SNAKE_CASE for constants.

Alternative concept: replacing constexpr and enum class with #define macros can reduce typed declarations but weakens type checking and scope safety. A macro generally occupies no storage; a constexpr constant often needs no runtime storage when optimised. Runtime complexity is effectively unchanged (O(1)); the main loss is compile-time

safety and maintainability.



TASK 3 – Choose Data Types and Check Type Compatibility

Value	Suggested type	Reason
Fare / balance in paise	long long	Exact integer money and large accumulated totals.
Hour / zone index	int	Small whole-number values.
Journey count	long long or unsigned long long	Supports large counts.
Active / peak flag	bool	Only true or false.
Passenger initial / code	char	One character, if required.
Percentage	int	Whole percentage values.
Display-only average	double	Useful for non-financial display or statistics.
Inferred local value	auto	Lets compiler infer a type from the initialiser.

On a typical 64-bit g++ system, sizeof(int) is commonly 4 bytes, sizeof(long long) is commonly 8 bytes, sizeof(double) is commonly 8 bytes, sizeof(bool) is commonly 1 byte and sizeof(char) is 1 byte. These are typical values, not universal guarantees; run sizeof on the compiler used for submission.

```
std::cout << sizeof(int) << '\\n';
std::cout << sizeof(long long) << '\\n';
std::cout << sizeof(double) << '\\n';
std::cout << sizeof(bool) << '\\n';
std::cout << sizeof(char) << '\\n';
```

Why integer paise?

A double stores many decimal fractions approximately in binary. Repeated calculations can create tiny rounding differences. Storing Rs 10.00 as 1000 paise and Rs 30.00 as 3000 paise allows exact addition and subtraction using integers.

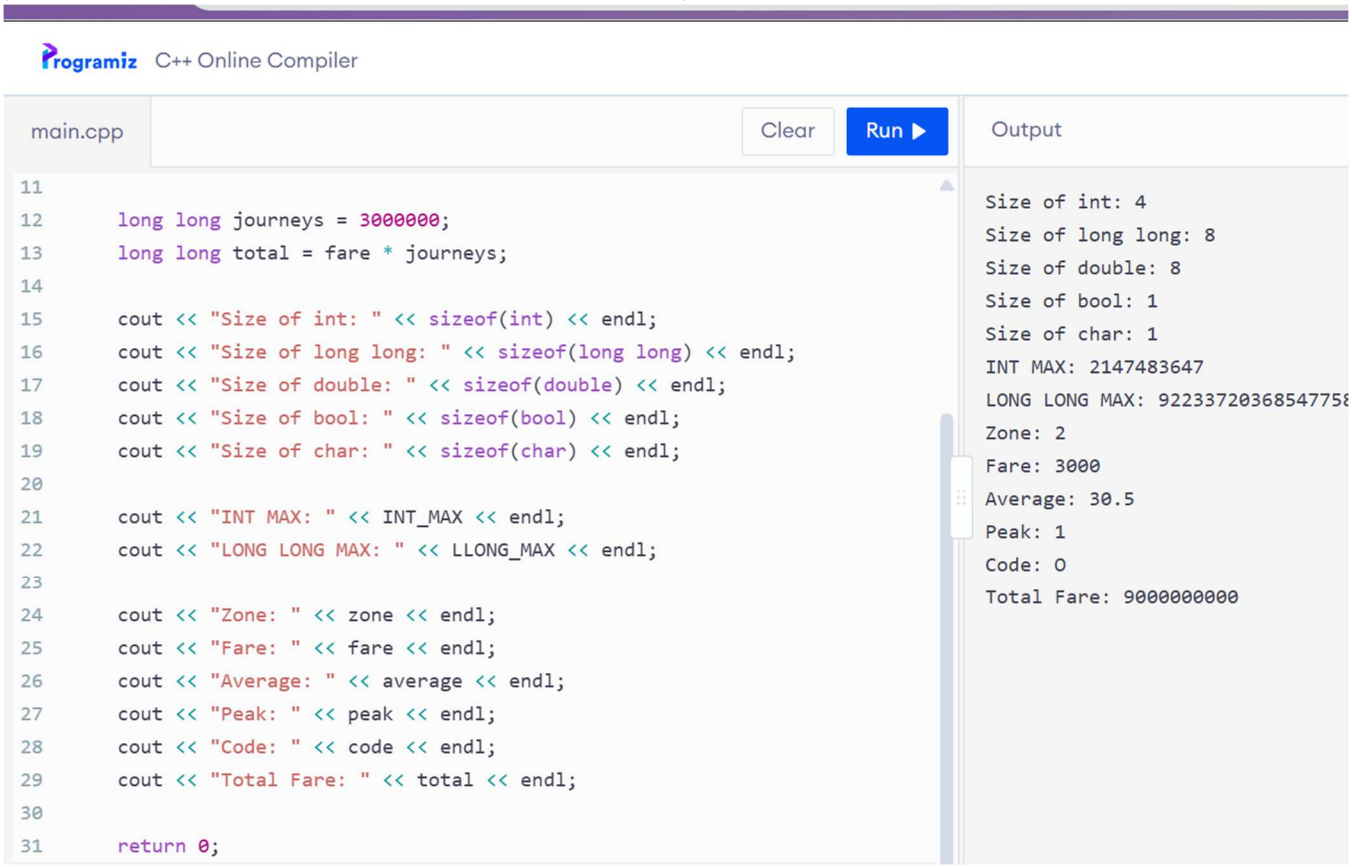
Type compatibility

- When int and double are used in one arithmetic expression, the int is normally converted to double.
- When a numeric value is assigned to bool, zero becomes false and a non-zero value becomes true.
- When a double is assigned to int without an explicit conversion, the fractional part is discarded.
- Mixing signed and unsigned integers can convert the signed value to unsigned, causing surprising comparisons.

Range and network collection

Use <climits> and print INT_MAX and LLONG_MAX on the target compiler. A 32-bit signed int commonly has maximum 2,147,483,647. At 3,000,000 journeys with a maximum base fare of Rs 30, the total is Rs 90,000,000 or 9,000,000,000 paise. The paise total exceeds the usual int range, so long long is required.

Alternative concept: double rupees and long long paise are commonly both 8 bytes and each addition is O(1). However, integer paise is exact for these fare amounts. A reproducible rounding demonstration is to add 0.1 as double repeatedly and compare with integer tenths; the double result may not equal the mathematically exact decimal total. The size and precision behaviour should be checked on the student's compiler.



TASK 4 – Control Implicit Conversions

Implicit conversion happens automatically when C++ changes a value from one type to another.

- Integer division: 7 / 2 is 3 because both operands are int. Assigning the result to double gives 3.0, not 3.5.
- Integer-to-double: static_cast<double>(a) / b converts a before division, producing 3.5.
- Char promotion: a char used in arithmetic is promoted to int; 'A' + 2 prints C, and ch + 1 prints its integer character code.
- Int-to-bool: assigning 5 to bool stores true. Default output is 1; std::boolalpha prints true.
- Signed/unsigned: comparing -1 with unsigned 1 converts -1 to a large unsigned value, so the comparison s < u is true on the usual unsigned conversion rules.
- Narrowing: int x{3.7}; is rejected because brace initialisation prevents a narrowing conversion.

Task 4 – predicted output (write prediction before execution)

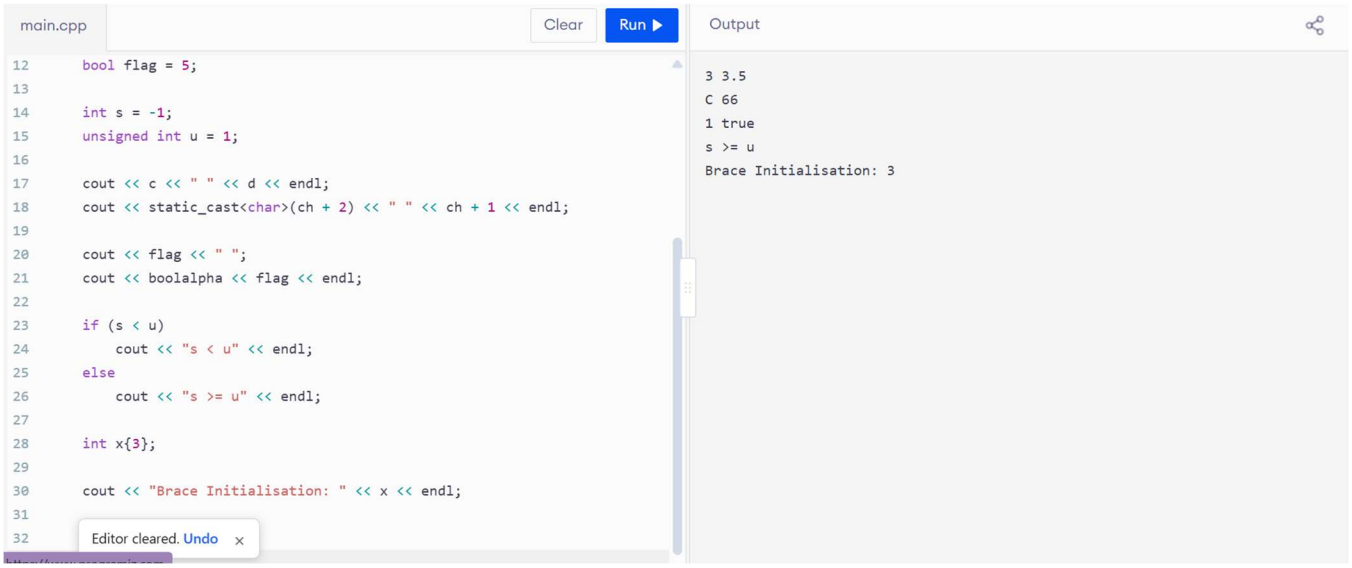
```
3 3.5
C 68
1 true
s >= u
```

Explanation: c receives the result of integer division (3). d uses an explicit conversion before division (3.5). ch is 'C'; ch + 1 is integer 68. bool flag stores true, printed first as 1 and then as true. In the signed/unsigned comparison, s becomes a large unsigned value, therefore s < u is false and the conditional prints s >= u.

Safer code

```
double exactDivision = static_cast<double>(a) / b;
int wholeNumber{3};
// int invalidValue{3.7}; // compile-time narrowing error
```

Alternative concept: using explicit static_cast and brace initialisation makes conversions visible and catches many narrowing mistakes at compile time. Casts do not normally change the asymptotic time or memory use; they improve clarity and compile-time checking. They cannot make an unsafe conversion logically correct by themselves.



TASK 5 – Write Correct Expressions with the Right Precedence

```
bool peak = (hour >= 8 && hour < 10) ||
            (hour >= 17 && hour < 20);
long long surcharge = peak ? baseFare * 20 / 100 : 0;
long long finalFare = baseFare + surcharge;
balance -= finalFare;
```

The && operator is evaluated before ||. Parentheses make the two peak windows easy to read. The conditional operator ?: chooses one of two expressions. Compound assignment balance -= fare is equivalent to balance = balance - fare.

- x++ returns the old value and then increments x; ++x increments first and returns the new value.
- i = i++ should be avoided because it is confusing and may have undefined or version-dependent behaviour depending on the expression context; write i++ or ++i as a separate statement.
- For integer operands, 10 / 3 is 3 and 10 % 3 is 1. In C++, -10 / 3 is -3 and -10 % 3 is -1 (remainder follows truncation toward zero).
- The expression with explicit parentheses is easier to read. The unparenthesised expression is equivalent because && has higher precedence than ||.

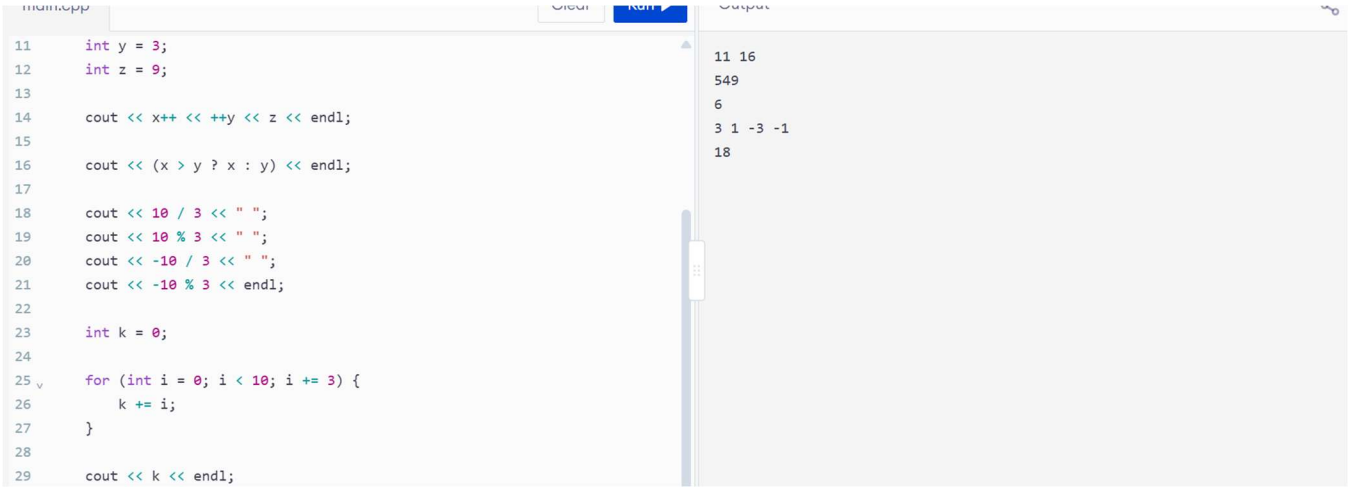
Task 5 – predicted output (write prediction before execution)

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Line 1: multiplication occurs before addition (5 + 3*2 = 11); parentheses force addition first in the second expression ((5+3)*2 = 16). Line 2: x++ contributes 5 and then x becomes 6; ++y first makes y 4 and contributes 4; z is 9. Line 3: x >

y is true, so 6 is printed; x % 4 * 2 is evaluated left to right for equal-precedence multiplicative operators: 6 % 4 * 2 = 4. Line 4: integer division and remainder produce 3, 1, -3, -1. Line 5: i takes 0, 3, 6, 9; k becomes 18.

Alternative concept: a bool peak[24] lookup table can replace the compound expression. Both checks are O(1). The table uses 24 bool entries (typically about 24 bytes, but sizeof should verify). It may reduce comparisons but adds a small fixed memory cost and must be initialised correctly.



```
11      int y = 3;
12      int z = 9;
13
14      cout << x++ << ++y << z << endl;
15
16      cout << (x > y ? x : y) << endl;
17
18      cout << 10 / 3 << " ";
19      cout << 10 % 3 << " ";
20      cout << -10 / 3 << " ";
21      cout << -10 % 3 << endl;
22
23      int k = 0;
24
25      for (int i = 0; i < 10; i += 3) {
26          k += i;
27      }
28
29      cout << k << endl;
```

```
11 16
549
6
3 1 -3 -1
18
```

TASK 6 – Design the Control Structures

Use if–else for validation and minimum-balance decisions. Use switch for the small, fixed set of zones. Use ?: for a short choice such as adding a surcharge. Use a range-based for loop when the journeys are already stored in an array or container; use while when the number of inputs is not known in advance. Use continue to skip an invalid journey and break to stop processing when required. A switch default case handles an unknown zone.

Algorithmic pseudocode

```
ALGORITHM CalculateMetroFare
INPUT zone, hour, balancePaise
SET minimumBalancePaise <- 5000
IF hour < 0 OR hour > 23 THEN
    OUTPUT "Invalid hour"
    STOP
END IF

SWITCH zone
    CASE Inner: SET baseFare <- 1000
    CASE Middle: SET baseFare <- 2000
    CASE Outer: SET baseFare <- 3000
    DEFAULT:
        OUTPUT "Invalid zone"
        STOP
END SWITCH

SET peak <- (hour >= 8 AND hour < 10) OR
            (hour >= 17 AND hour < 20)
IF peak THEN
    SET fare <- baseFare + (baseFare * 20 / 100)
ELSE
    SET fare <- baseFare
END IF

IF balancePaise - fare >= minimumBalancePaise THEN
    SET balancePaise <- balancePaise - fare
```

```
    OUTPUT fare and balancePaise
ELSE
    OUTPUT "Denied: low balance"
END IF
END ALGORITHM
```

Completed TODO logic

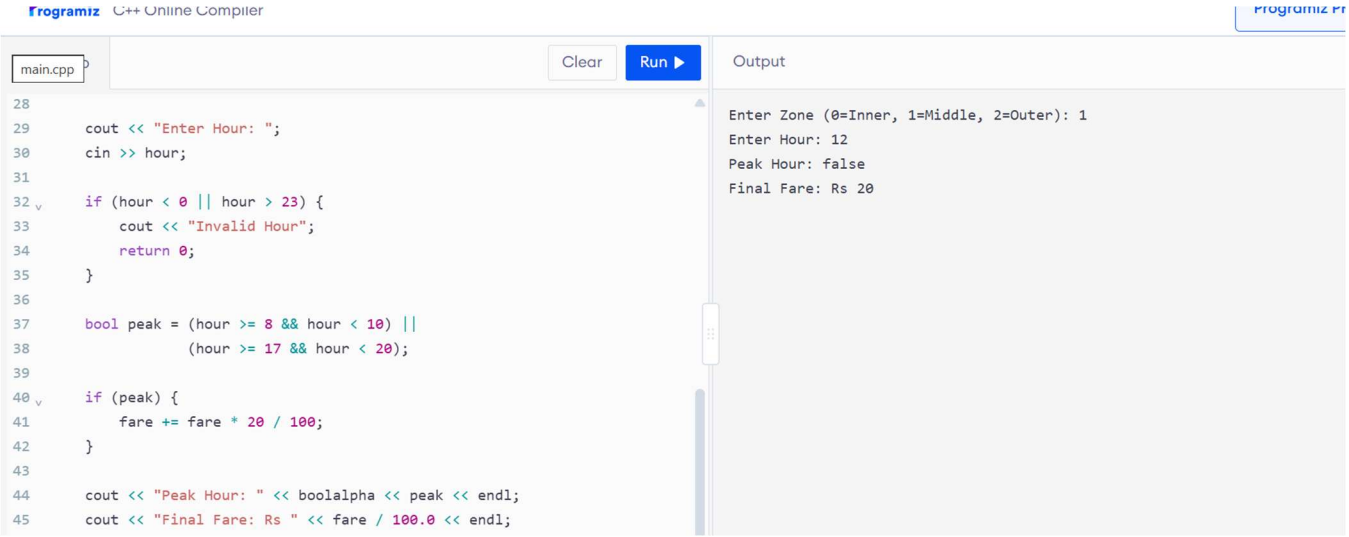
```
long long fare(Zone z, bool peak)
{
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default: return -1;
    }
    if (peak) {
        base = base + base * PEAK_SURCHARGE_PERCENT / 100;
    }
    return base;
}

bool canTravel(long long balance, long long f)
{
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}
```

Expected program output from the assignment's supplied main():

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

Boundary check: 10:00 AM and 8:00 PM are off-peak. 8:00 AM and 5:00 PM are peak. Alternative: an array lookup for base fare is O(1), like switch. The array uses three fare values; adding a fourth zone requires extending the array and validation logic. A switch requires adding another case.



TASK 7 – From Procedural to Object-Oriented

Procedural version

```
long long balancePaise = 10000;
const long long minimumPaise = 5000;
```

```
bool canTravel(long long balance, long long fare)
{
    return fare >= 0 && balance - fare >= minimumPaise;
}

// Use:
// if (canTravel(balancePaise, farePaise))
//     balancePaise -= farePaise;
```

Object-oriented version

```
class SmartCard {
private:
    long long balancePaise;
    const long long minimumPaise = 5000;

public:
    SmartCard(long long initialBalance) {
        balancePaise = initialBalance;
    }

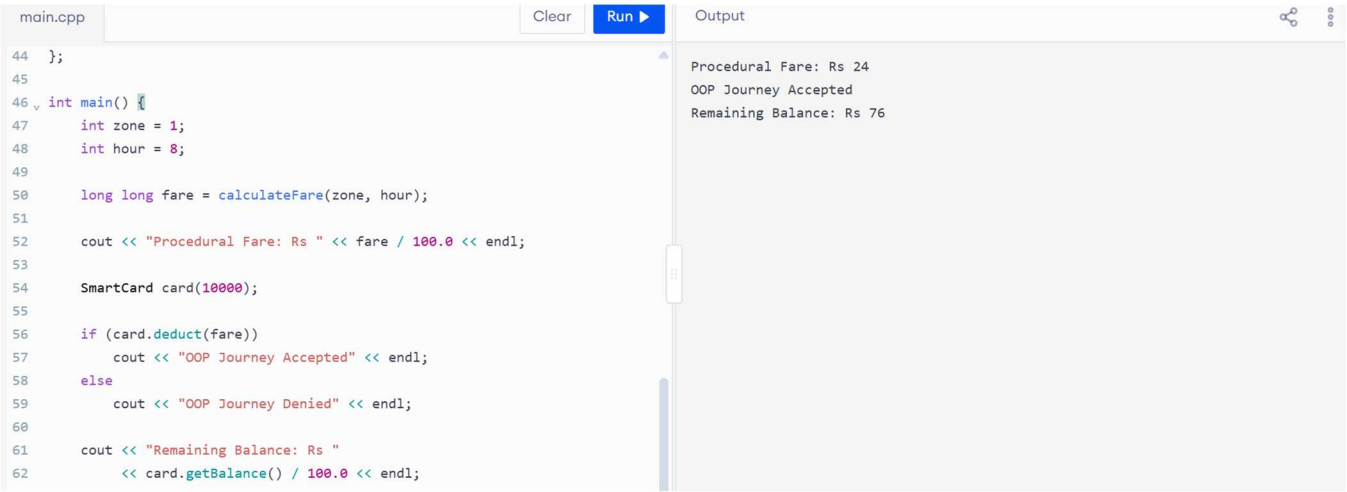
    bool canTravel(long long farePaise) const {
        return farePaise >= 0 &&
               balancePaise - farePaise >= minimumPaise;
    }

    bool deduct(long long farePaise) {
        if (!canTravel(farePaise)) {
            return false;
        }
        balancePaise -= farePaise;
        return true;
    }

    long long getBalance() const {
        return balancePaise;
    }
};
```

The procedural version can work correctly if every function follows the rule, but any function can accidentally change the global balance. In the OOP version, balancePaise is private and only deduct() changes it after canTravel() succeeds. This prevents a negative or below-minimum balance through the public interface.

If the minimum balance changes, the OOP version changes the class constant in one place. A public balance member would allow other code to set an invalid value directly. Time and space remain O(1) in either approach; encapsulation changes safety and maintainability, not Big-O.



TASK 8 – Read and Print Data Safely

```
int hour;
std::cout << "Enter hour (0-23): ";
if (!(std::cin >> hour)) {
    std::cin.clear();
    std::cin.ignore(10000, '\\n');
    std::cout << "Invalid input. Enter a number.\\n";
} else if (hour < 0 || hour > 23) {
    std::cout << "Hour must be between 0 and 23.\\n";
} else {
    std::cout << "Accepted hour: " << hour << '\\n';
}
```

cin.fail() or the failed state of cin detects letters entered where a number is expected. cin.clear() resets the error flags. cin.ignore() discards the invalid characters up to the end of the line. The program should validate the numeric range after successful input.

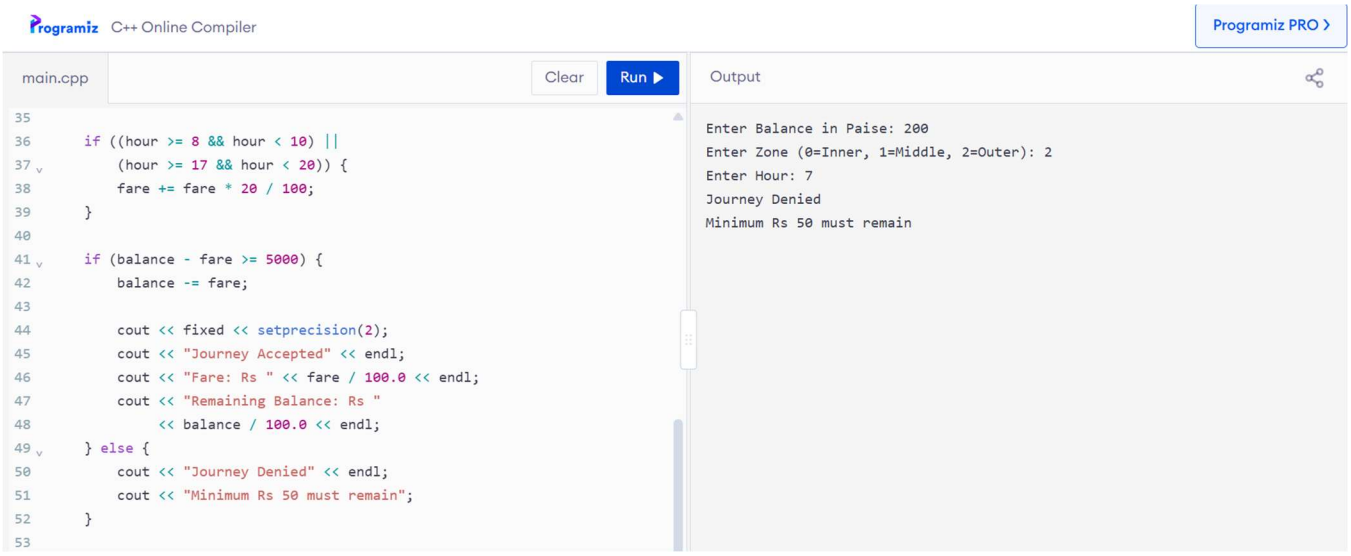
Formatting money

```
std::cout << "Fare Rs " << farePaise / 100 << '.'
          << std::setw(2) << std::setfill('0')
          << farePaise % 100 << std::setfill(' ') << '\\n';
```

iomanip tools: setw sets the width for the next field, setfill chooses padding, fixed and setprecision control decimal output when printing floating-point values. For integer paise, printing rupees and the two-digit remainder avoids floating-point formatting errors.

A neat report should show journey number, zone, time, peak/off-peak, base fare, surcharge, final fare, status and remaining balance.

Alternative concept: getline() reads a complete line and stoi() converts it inside try/catch. Processing a line of length L takes O(L) time and O(L) buffer space. cin >> is convenient for simple numeric input; getline + stoi provides clearer control over the entire line but needs exception handling and full-line validation.



TASK 9 – Implement and Test the Fare Logic

The following test table uses the base fares and peak rule from the scenario. All amounts are in rupees for readability. Concessions are excluded because the assignment does not state their rates.

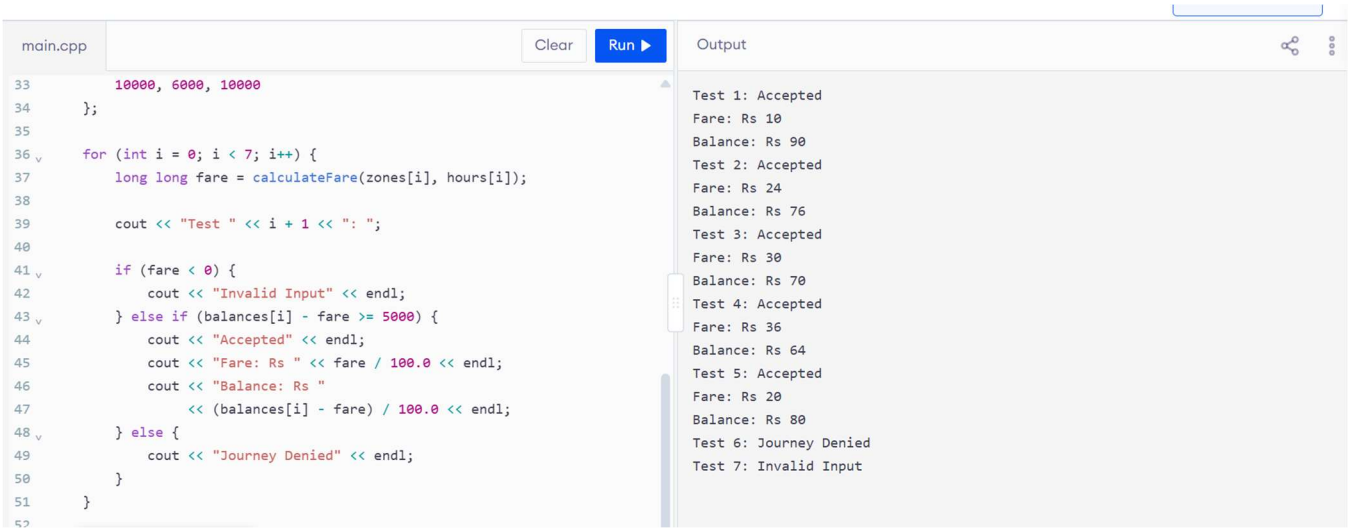
Test	Input	Expected result	Reason
T1	Inner, 07:59, balance Rs 100	Fare Rs 10; balance Rs 90	Off-peak Inner.
T2	Middle, 08:00, balance Rs 100	Fare Rs 24; balance Rs 76	Peak 20% surcharge.
T3	Outer, 10:00, balance Rs 100	Fare Rs 30; balance Rs 70	10:00 is off-peak.
T4	Outer, 17:00, balance Rs 100	Fare Rs 36; balance Rs 64	17:00 is peak.
T5	Middle, 20:00, balance Rs 100	Fare Rs 20; balance Rs 80	20:00 is off-peak.
T6	Inner, 08:00, balance Rs 60	Denied; balance Rs 60	Rs 50 minimum would not remain after Rs 12 fare.
T7	Inner, 09:00, balance Rs 62	Accepted; balance Rs 50	Exactly Rs 50 remains.
T8	Zone = 9	Invalid zone	Unknown zone rejected.
T9	Recharge = -500 paise	Rejected	Negative recharge is invalid.
T10	Hour input = abc	Rejected and re-prompted	Non-numeric input handled safely.

Five-journey hand calculation

Journey	Zone / time	Fare	Balance after journey (starting Rs 200)
1	Inner, 07:00	Rs 10.00	Rs 190.00
2	Middle, 08:30	Rs 24.00	Rs 166.00
3	Outer, 12:00	Rs 30.00	Rs 136.00
4	Outer, 18:00	Rs 36.00	Rs 100.00
5	Inner, 20:00	Rs 10.00	Rs 90.00
Total	5 journeys	Rs 110.00	Rs 90.00 remaining

Complexity: calculating one fare is O(1), validating one journey is O(1), and processing n journeys is O(n). The test table uses O(t) time to execute t tests and O(t) space to store them. A table makes new boundary tests easier to add.

Execution evidence to attach: compile screenshot, run screenshot, and actual result column verified on the student's own compiler. The expected values above are calculated from the assignment rules; do not label them as observed compiler output until you execute the code.



TASK 10 – Compare Expectations with Execution

To compare network totals, assume every journey is an Outer-zone off-peak journey costing Rs 30. This is a deliberately simple upper-base-fare workload, not a prediction of the actual passenger mix.

Scale	Journeys/day	Total Rs	Total paise	Fits usual 32-bit int paise?
One station	20,000	600,000	60,000,000	Yes
One line	500,000	15,000,000	1,500,000,000	Yes, but close to limit
Whole network	3,000,000	90,000,000	9,000,000,000	No

A usual 32-bit signed int maximum is 2,147,483,647. The whole-network total of 9,000,000,000 paise exceeds it, so signed int overflows. A long long is the appropriate accumulator. A double can represent large totals but decimal binary fractions can cause rounding for values such as 0.1; for fare totals, integer paise is safer.

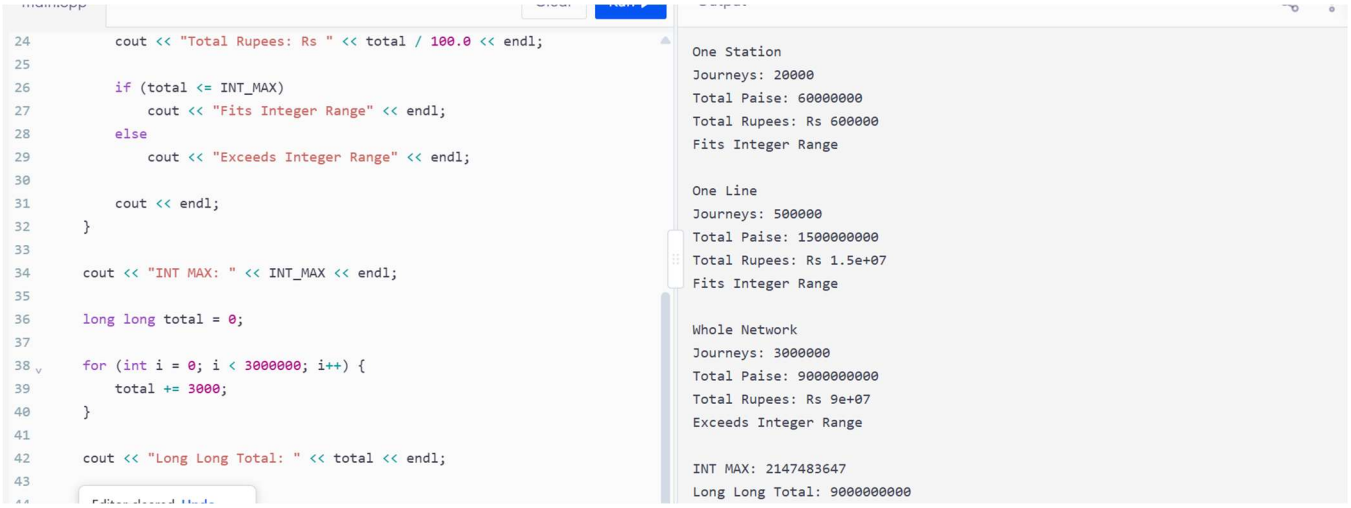
Accumulator	Typical size	At 3,000,000 × 3000 paise	Conclusion
int	4 bytes	Exceeds usual signed int maximum	Unsafe
long long	8 bytes	9,000,000,000 paise	Safe for this scenario
double	8 bytes	Can hold the magnitude; decimal fractions may round	Avoid for exact money

Each addition is O(1) for all three types. Space per accumulator is O(1). The exact byte sizes and compiler warnings must be recorded from the compiler used. Compile with: g++ -std=c++17 -Wall -Wextra -Wconversion main.cpp -o metro

Common warning categories to inspect: signed/unsigned comparison, narrowing or conversion warnings, unused variables and missing return paths. The exact warnings depend on the code and compiler; include the actual warning text from your own build rather than claiming a warning that was not produced.

Expected versus actual evidence: record predicted total, program total, difference, and explanation. For integer-paise calculations with valid integer fares, the expected and actual totals should match exactly. For double accumulation of decimal fractions, any difference should be measured by running a separate experiment.

Rounding experiment pseudocode: set double total = 0; repeat n times: total += 0.1; compare with n / 10.0 and with integer tenths n. This demonstrates that binary floating point may not store 0.1 exactly.



```
        cout << "Total Rupees: Rs " << total / 100.0 << endl;

        if (total <= INT_MAX)
            cout << "Fits Integer Range" << endl;
        else
            cout << "Exceeds Integer Range" << endl;

        cout << endl;
    }

    cout << "INT MAX: " << INT_MAX << endl;

    long long total = 0;

    for (int i = 0; i < 3000000; i++) {
        total += 3000;
    }

    cout << "Long Long Total: " << total << endl;
```

```
One Station
Journeys: 20000
Total Paise: 60000000
Total Rupees: Rs 600000
Fits Integer Range

One Line
Journeys: 500000
Total Paise: 1500000000
Total Rupees: Rs 1.5e+07
Fits Integer Range

Whole Network
Journeys: 3000000
Total Paise: 9000000000
Total Rupees: Rs 9e+07
Exceeds Integer Range

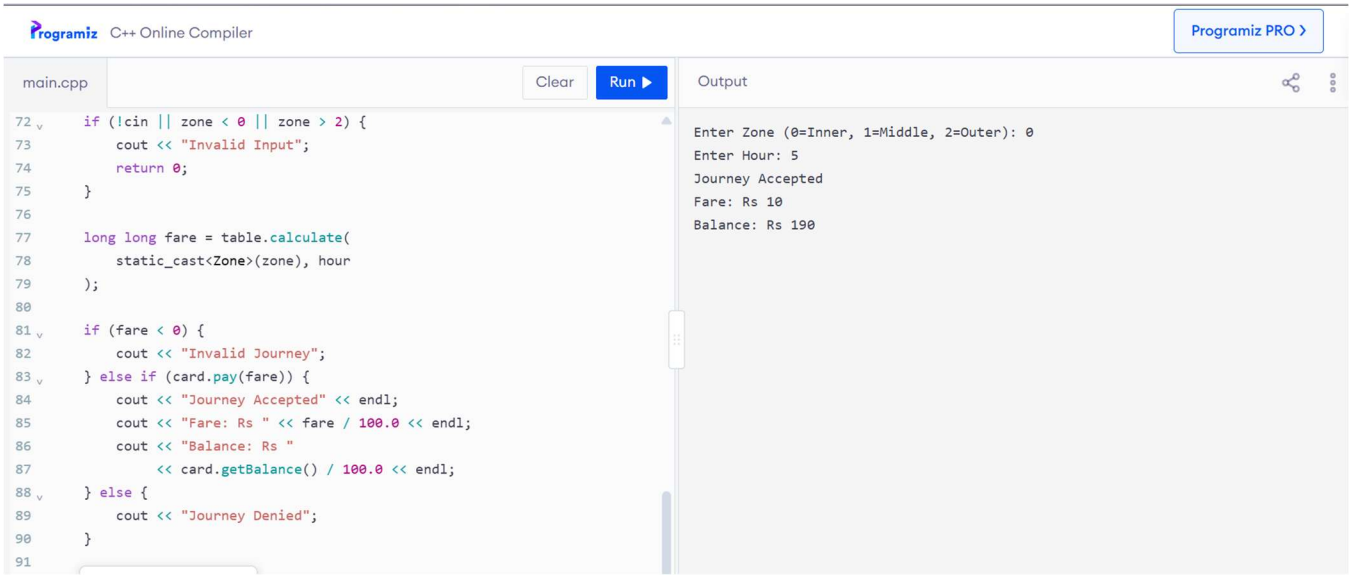
INT MAX: 2147483647
Long Long Total: 9000000000
```

TASK 11 – Recommend and Defend the Final Design

Recommended design: use an OOP structure with SmartCard, Journey, Passenger and FareTable classes. Store all money in long long paise. Use constexpr constants for fixed values, enum class for zones, integer arithmetic for percentages, and safe input validation.

- Types: long long for balances and accumulated fares; int for hour, zone index and percentages; bool for peak and active flags.
- Constants: base fares, 20% peak surcharge and Rs 50 minimum balance must be declared once.
- Classes: SmartCard protects the balance; FareTable centralises rules; Journey stores trip details; Passenger stores passenger category.
- Fare revision: change one fare-table value and run the full regression test table. If the fare table is loaded from a configuration file, validate the file at startup.
- Limitations: concession rates are not specified; the basic design assumes one fare per journey based on the farthest zone; it does not yet model transfers, refunds, card expiry or concurrent gate transactions.
- Network scale: O(1) fare processing per journey and O(n) for n journeys. long long avoids the demonstrated int total overflow.

Alternative concept: load z fare values from a configuration file at startup. Reading the file costs O(z) time and O(z) memory for z zone fares. This avoids recompiling for a fare revision but requires file validation, error handling and secure configuration management. Compile-time constexpr values use very little runtime overhead but require rebuilding and redeploying after a change. For a real metro, a validated configuration/database-backed FareTable is more flexible.



TASK 12 – Re-evaluate and Adapt the Design

Changed requirements: add Airport zone at Rs 60, a 10% weekend discount that does not combine with peak surcharge, and a monthly pass for unlimited journeys in two chosen zones.

A. Impact analysis (2 marks)

- Add Zone::Airport and an Airport fare constant of 6000 paise.
- Expand the fare table from three entries to four entries.
- Update switch statements, zone validation, input menu, test cases and any UML diagrams.
- Add a weekend flag or date/day value and a 10% weekend discount rule.
- Define the precedence rule: on weekends apply the 10% discount instead of the 20% peak surcharge; do not apply both.
- Add a Pass class with passId, validFrom, validTo, firstZone, secondZone and active status.
- Update Journey to check whether the passenger's valid pass covers the journey zone.

B. Evaluate original design (2 marks)

The original three-case switch can handle Airport only after adding another case and extending validation. The original SmartCard can still store money, but it does not by itself represent a monthly pass. A fare table indexed by zone and a separate discount rule are easier to extend. The original design therefore needs modification rather than being retained unchanged.

Worked example: Airport base fare is Rs 60. On a weekday peak hour, fare = Rs 60 + 20% = Rs 72. On a weekend peak hour, apply only the weekend discount: Rs 60 − 10% = Rs 54. It is not Rs 64.80 because the two adjustments must not be combined.

C. Adapted design and code (2 marks)

```
enum class Zone { Inner, Middle, Outer, Airport };
constexpr long long FARES[] = {1000, 2000, 3000, 6000};

long long revisedFare(Zone zone, bool peak, bool weekend)
{
    int index = static_cast<int>(zone);
    if (index < 0 || index >= 4) return -1;

    long long amount = FARES[index];
```

```
    if (weekend) {
        amount = amount - amount * 10 / 100;
    } else if (peak) {
        amount = amount + amount * 20 / 100;
    }
    return amount;
}

class Pass {
private:
    int firstZone;
    int secondZone;
    bool active;

public:
    Pass(int z1, int z2) : firstZone(z1), secondZone(z2), active(true) {}

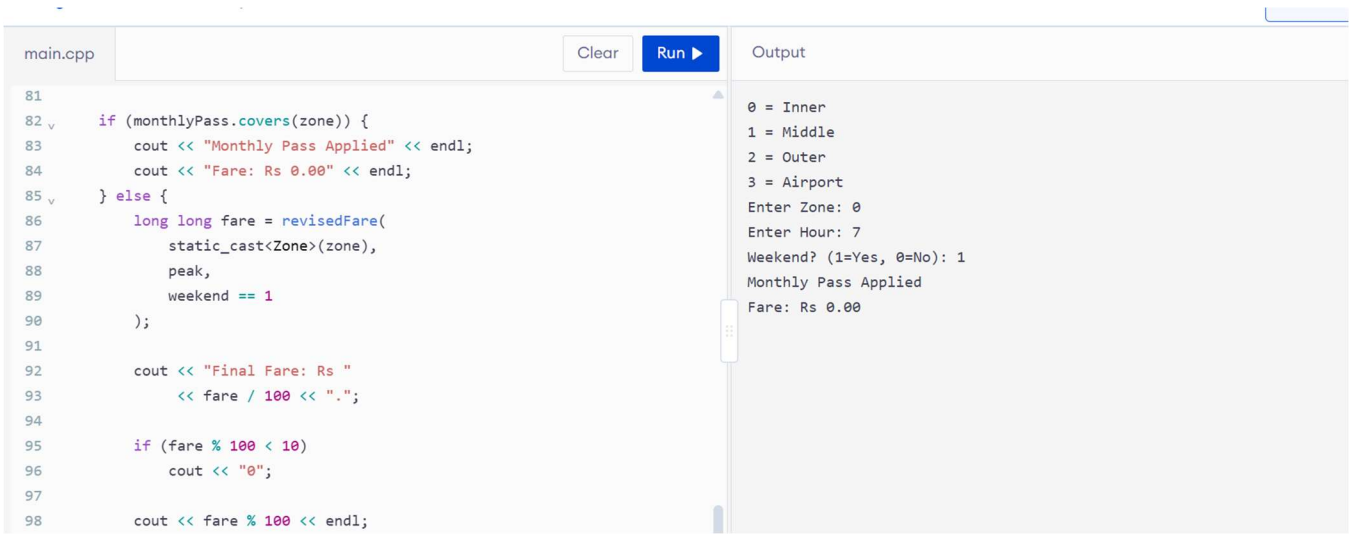
    bool covers(int zone) const {
        return active && (zone == firstZone || zone == secondZone);
    }
};
```

Monthly pass rule: if the pass is valid for the current month and the journey zone matches either selected zone, the fare is Rs 0 for that journey. Otherwise, calculate the normal fare using the weekend/weekday rule. The exact policy for a journey spanning more than one zone must be confirmed with the metro/faculty; this example treats the fare zone as the farthest zone and pass coverage as an exact match.

D. Evidence-based defence

Test	Input	Expected result	Reason
D1	Airport weekday off-peak	Rs 60.00	Base fare.
D2	Airport weekday peak	Rs 72.00	20% surcharge.
D3	Airport weekend off-peak	Rs 54.00	10% discount.
D4	Airport weekend peak	Rs 54.00	Weekend discount replaces peak surcharge.
D5	Monthly pass covers Inner; Inner trip	Rs 0.00	Unlimited covered journey during pass validity.
D6	Monthly pass covers Inner; Outer trip	Normal Outer fare	Zone not covered by pass.

Defence: modify the original design by using a four-entry fare table, a separate weekend/weekday adjustment rule and a Pass class. This avoids duplicating fare expressions and makes future zones or pass rules easier to maintain. The trade-off is extra validation and state (pass dates and selected zones). Run the tests above and attach screenshots to demonstrate the actual implementation.



Overall Conclusion

The metro fare system should use object-oriented design, integer paise, well-named constexpr constants, enum class zones, explicit conversion where needed, readable control expressions and safe input validation. The minimum-balance rule must be enforced before deduction. Testing must include normal journeys, time boundaries, invalid inputs and large network totals. For the Day-4 change, a fare table plus a separate discount rule and Pass class provides a more extensible design than an unchanged switch-only solution.

C++ Program

```
#include <iostream>
#include <iomanip>
#include <limits>
#include <string>

enum class Zone { Inner, Middle, Outer };

constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_PERCENT = 20;
constexpr long long MIN_BALANCE = 5000;

bool isPeak(int hour)
{
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
}

long long calculateFare(Zone zone, int hour)
{
    long long base = 0;

    switch (zone) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default: return -1;
    }

    if (isPeak(hour)) {
        base = base + base * PEAK_PERCENT / 100;
    }
    return base;
}

class SmartCard {
private:
    long long balancePaise;

public:
    SmartCard(long long initialBalance) : balancePaise(initialBalance) {}

    bool canTravel(long long farePaise) const
```

```
    {
        return farePaise >= 0 &&
               balancePaise - farePaise >= MIN_BALANCE;
    }

    bool deduct(long long farePaise)
    {
        if (!canTravel(farePaise)) {
            return false;
        }
        balancePaise -= farePaise;
        return true;
    }

    long long getBalance() const
    {
        return balancePaise;
    }
};

void printMoney(long long paise)
{
    std::cout << "Rs " << paise / 100 << '.'
              << std::setw(2) << std::setfill('0')
              << paise % 100 << std::setfill(' ');
}

int main()
{
    SmartCard card(20000);
    int zoneInput;
    int hour;
    int journey = 1;

    while (journey <= 5) {
        std::cout << "Journey " << journey
                  << " (0=Inner, 1=Middle, 2=Outer): ";

        if (!(std::cin >> zoneInput)) {
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << "Invalid zone input. Try again.\n";
            continue;
        }

        std::cout << "Enter hour (0-23): ";
        if (!(std::cin >> hour)) {
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << "Invalid hour input. Try again.\n";
            continue;
        }
```

```
        if (zoneInput < 0 || zoneInput > 2 || hour < 0 || hour > 23) {
            std::cout << "Invalid zone or hour. Journey not processed.\n";
            continue;
        }

        Zone zone = static_cast<Zone>(zoneInput);
        long long fare = calculateFare(zone, hour);

        if (card.deduct(fare)) {
            std::cout << "Accepted. Fare ";
            printMoney(fare);
            std::cout << ", balance ";
            printMoney(card.getBalance());
            std::cout << '\n';
        } else {
            std::cout << "Denied: minimum balance rule. Balance ";
            printMoney(card.getBalance());
            std::cout << '\n';
        }
        ++journey;
    }
    return 0;
}
```